Annexure A

ARTICLE REVIEW

CSA1711-ARTIFICIAL INTELLIGENCE

CO4-ASSESSMENT TOOL 3

Submission Details

Selected Article: A Decision Tree Approach for Enhancing Real-Time Response in Exigent Healthcare Unit Using Edge Computing

Authors: Eram Fatima Siddiqui, Tasneem Ahmed, Sandeep Kumar Nayak

Journal: Measurement: Sensors (Elsevier), Volume 32, Article 100979

Year: 2024

DOI: 10.1016/j.measen.2023.100979

Database: Elsevier — ScienceDirect (indexed, Elsevier Digital Library)

Topic Area: Decision Trees / Inductive Learning (CO 4)

Note: Full-text access on ScienceDirect requires institutional/subscription login; the abstract, indexing record, and citing literature (used to verify the citation, scope, and comparative claims below) are openly accessible. Findings reported in this review are drawn from the publicly available abstract and record.

Task 1: Article Summary

(a) Full Citation and Problem Domain

Citation (APA-style): Siddiqui, E. F., Ahmed, T., & Nayak, S. K. (2024). A decision tree approach for enhancing real-time response in exigent healthcare unit using edge computing. Measurement: Sensors, 32, 100979. https://doi.org/10.1016/j.measen.2023.100979

Domain / Application Area: IoT-based smart healthcare, combined with Mobile Edge Computing (MEC). The work sits at the intersection of remote patient monitoring, edge/fog computing infrastructure, and applied machine learning.

Specific ML Problem Addressed: Multi-class classification of a patient's health severity (Low Risk / Normal / High Risk) from real-time bio-sensor readings and historical medical records, followed by a downstream task-offloading decision (No Offloading / Edge Offloading / Collaborative Edge Offloading) that is driven by the classifier's output. In essence, the paper uses a Decision Tree as the triage mechanism that then dictates how urgently and where the associated computation should be processed.

(b) Objective and Research Gap (Summary in Own Words)

Modern IoT-based healthcare systems continuously stream vital-sign data from wearable and bedside bio-sensors, but sending every reading to a distant cloud server for analysis introduces latency that can be dangerous when a patient's condition is deteriorating. The authors set out to close this gap by placing decision-making closer to the patient. They train a Decision Tree on medical parameters and historical health records so that incoming sensor data can be classified into a risk category almost instantly at the network edge, rather than waiting on round-trip cloud communication. The identified research gap is that prior fog/edge healthcare frameworks (such as energy-aware IoMT-to-fog scheduling, latency-optimised fog computing, and multimedia data-segregation approaches) treat task offloading and patient urgency as loosely connected problems, optimising system-level metrics like energy or latency without tightly coupling the offloading decision to the patient's actual clinical severity. The authors fill this gap by making the Decision Tree's severity classification the direct trigger for the offloading strategy: only when a patient is categorised as high risk does the system escalate to Edge or Collaborative Edge Offloading, ensuring urgent cases get prioritised computational and communication resources while low-risk cases are handled locally. This tightens the link between clinical need and system resource allocation, which the authors report yields a substantial improvement in overall system performance compared to the three benchmark schemes.

Task 2: Methodology Analysis

(a) ML Technique and Dataset

Technique used: A supervised Decision Tree classifier, trained offline on labelled patient data and then deployed for real-time inference at the network edge. The tree evaluates medical parameters (e.g., readings gathered from multiple Bio Sensors — BS — such as heart rate, temperature, and related vitals) against learned thresholds at each internal node, following root-to-leaf splits until it outputs one of three severity classes: Low Risk, Normal, or High Risk. This severity label is then used as the input condition for a separate offloading-decision module that selects among No Offloading, Edge Offloading, and Collaborative Edge Offloading.

Dataset: The model is trained and evaluated on medical parameter data collected through multiple Bio Sensors together with historical Medical Health Records (MHR) of patients, consistent with the standard IoT-healthcare pipeline the paper builds on. The publicly visible abstract and record do not disclose the exact sample size, feature list, or public dataset name, and the authors' data-availability statement indicates the data is available on request rather than published as an open dataset. Preparation would typically involve sensor-signal cleaning, feature extraction from the raw BS streams, and labelling of risk categories from the MHR before the tree is trained — this level of preprocessing detail was not confirmable from the openly accessible portions of the article.

(b) Mapping to CO 4 Topics

CO 4 Topic	Relevance to the Article
Decision Trees	The core classifier is a trained Decision Tree that partitions patients into Low Risk / Normal / High Risk categories using medical parameter thresholds at each node — a direct application of hierarchical, rule-based classification.

Inductive Learning	The tree is built inductively from historical Medical Health Records and real-time bio-sensor readings, generalising labelled examples of patient risk categories into decision rules that are then applied to unseen streaming data.
Statistical Learning Methods	Related indirectly: the severity-classification stage functions analogously to a multi-class statistical classifier, and the paper's evaluation approach (comparative benchmarking against alternative schemes) mirrors standard statistical-learning validation practice.
Reinforcement Learning	Not used in this article — task offloading decisions are rule-driven from Decision Tree outputs rather than learned via a reward-based RL agent; this is flagged in the Reflection section as a natural extension.

Methodological Strength: Using a Decision Tree is well-suited to this setting because the model is lightweight, interpretable, and cheap to evaluate at inference time — critical properties for resource-constrained edge devices where a deep neural network would be too costly to run under strict latency budgets. The interpretability also matters clinically: a clear if-then path from vitals to risk category is easier for medical staff to audit than a black-box prediction.

Methodological Limitation: A single Decision Tree is prone to overfitting and can be unstable — small changes in training data can produce quite different splits — which raises reliability concerns for a system making triage-like decisions. The paper's public abstract does not indicate whether pruning, cross-validation, or an ensemble alternative (e.g., Random Forest) was benchmarked to mitigate this, nor does it report class-imbalance handling for what is likely a skewed distribution (most sensor readings correspond to Normal/Low-Risk patients, with High-Risk events comparatively rare).

Task 3: Results & Critical Evaluation

(a) Key Results

The authors report the results primarily as a system-level comparison rather than as classifier-only metrics (accuracy/F1/AUC-ROC figures for the Decision Tree stage itself are not stated in the openly available abstract). The headline finding is that the proposed Decision-Tree-driven offloading framework outperforms three benchmark schemes with an overall improvement of approximately 88% in system performance.

Method	Approach Type	Primary Metric Reported	Relative Outcome
Proposed Decision Tree–based MEC offloading model	Trained Decision Tree classifier + rule-based offloading (No Offload / Edge Offload / Collaborative Edge Offload)	Overall system performance improvement	Best performing — ~88% improvement over baselines
Energy-Efficient IoMT to Fog Interoperability (Task Scheduling)	Fog-based task scheduling heuristic	System performance / latency	Lower than proposed method

Optimized Latency Fog Computing	Fog computing latency-optimization scheme	Latency-oriented performance	Lower than proposed method
Intelligent Multimedia Data Segregation	Data segregation-based offloading	System performance	Lower than proposed method

Because the granular per-class classification metrics (accuracy, precision/recall, F1-score, AUC-ROC) for the Decision Tree severity classifier were not visible in the openly accessible record, this review reports the system-level comparison the authors foreground in their abstract; a reader wanting the underlying confusion-matrix-level classifier performance would need to consult the full text via institutional access.

(b) Critical Evaluation of Results

- Realism of metrics: An aggregate '88% improvement in system performance' is a strong, attention-grabbing figure, but it is a composite metric whose components (e.g., weighted combination of latency, energy, or throughput gains) are not decomposed in the abstract. Without seeing the underlying formula, it is difficult to judge how realistic or reproducible the number is, and composite scores can sometimes overstate real-world gains by combining several favourable sub-metrics.
- Dataset/evaluation bias: The paper does not appear to rely on a large, publicly benchmarked dataset, and important details (sample size, class balance between risk categories, train/test split strategy) are not confirmable from the open record. If the High-Risk class is rare, a Decision Tree can achieve seemingly strong aggregate accuracy while still missing genuinely urgent cases — a serious concern in a healthcare-triage context.
- Evaluation protocol: It is unclear whether the comparison against the three baseline methods (IoMT-to-Fog task scheduling, Optimized Latency Fog Computing, Intelligent Multimedia Data Segregation) was performed on identical workloads/datasets or reproduced from each baseline's own reported figures — the latter would make the comparison less rigorous, since baselines were likely tuned and evaluated on their own original test conditions.
- Additional experiments that would strengthen the findings: (i) reporting standard classification metrics (accuracy, F1-score, confusion matrix) for the Decision Tree stage specifically; (ii) comparing the single Decision Tree against ensemble methods (Random Forest, Gradient Boosted Trees) and a simple statistical baseline (Logistic Regression/Naïve Bayes) to justify the choice of technique; (iii) testing under varying network conditions (bandwidth, jitter, number of concurrent patients) to see if the 88% figure holds under realistic edge-network stress; and (iv) a sensitivity/robustness analysis across different patient sub-populations.

(c) Real-World Applicability

The core idea — using a lightweight, interpretable classifier at the edge to triage patients and drive resource allocation — is directly deployable in remote patient monitoring, ICU tele-monitoring, ambulance/pre-hospital triage, and elderly home-care systems, where network latency to a distant cloud can be a genuine safety risk. Wearable-sensor vendors and hospital IoT gateways are natural deployment points for such a model.

Scaling challenges for industry deployment include: (1) Data availability — clinical-grade labelled datasets covering diverse patient demographics are hard to obtain and are subject to strict privacy regulation (HIPAA/GDPR-equivalent frameworks), which limits how broadly a single trained tree will generalise. (2) Compute and infrastructure — while a Decision Tree itself is cheap to run, building and maintaining the surrounding MEC/fog infrastructure (base stations, edge servers, orchestration for collaborative offloading) is a significant capital and operational investment for a hospital network. (3) Ethical and safety issues — an automated triage decision carries direct patient-safety consequences; false negatives (a High-Risk patient misclassified as Normal) are far costlier than false positives, so any real deployment would need conservative decision thresholds, human-in-the-loop verification, rigorous regulatory approval, and continuous model monitoring for drift as patient populations or sensor hardware change.

Task 4: Reflection & Learning Outcome

(a) Three Key Insights

- Insight 1 — Decision Trees as inductive learners under real constraints: Reading the article reinforced that Inductive Learning is not just a theoretical concept of generalising from examples to rules — here it becomes a practical engineering constraint, since the induced tree must be small and shallow enough to run within milliseconds on an edge device. This connects directly to the course's coverage of inductive bias and the trade-off between model complexity and generalisation.
- Insight 2 — Interpretability has operational, not just academic, value: The course frames Decision Trees as 'interpretable' models mainly for explainability to humans, but this article shows a second, more operational reason to prefer them: their shallow if-then structure keeps inference cost low enough for real-time deployment, which is a decisive factor in latency-critical systems where a neural network would be computationally too expensive.
- Insight 3 — A classifier's output can itself become a control signal: The severity label produced by the Decision Tree is not the end product; it is fed forward to drive a separate decision system (the offloading policy). This deepened my understanding that in applied ML pipelines, a classification model from CO 4 topics is often only the first stage of a larger decision-making system, foreshadowing how Reinforcement Learning agents could later learn the downstream offloading policy itself rather than following fixed rules.

(b) Proposed Extension

Proposed experiment: Replace the fixed rule-based offloading policy that currently sits downstream of the Decision Tree with a Reinforcement Learning agent (e.g., a tabular Q-learning or lightweight Deep Q-Network) that learns the No-Offload / Edge-Offload / Collaborative-Edge-Offload decision directly from a reward signal combining patient risk level, latency, energy consumption, and edge-server load — rather than a static mapping from severity class to offloading mode.

Justification and expected impact: This is feasible because the article already establishes the two core ingredients an RL formulation needs: a state representation (the Decision Tree's severity output plus network/edge-load parameters) and a discrete action space (the three offloading modes) that the system

already uses. Training could begin in a simulated edge environment (e.g., using an existing fog/edge simulator such as iFogSim or YAFS) before real deployment, keeping data and compute requirements modest. The expected impact is a more adaptive system: instead of a fixed rule ('High Risk always triggers Collaborative Edge Offloading'), the RL agent could learn to weigh real-time network congestion and edge-server availability against patient urgency, potentially improving both response time and resource efficiency beyond the reported 88% baseline improvement, while also directly extending the Decision Tree/Inductive Learning technique studied in the article into the Reinforcement Learning topic area of CO 4.

End of Review